

SUBMITTED TO: DR NAVEED AKTAR



DATA STRUCTURES AND ALGORITHMS

BOGGLE

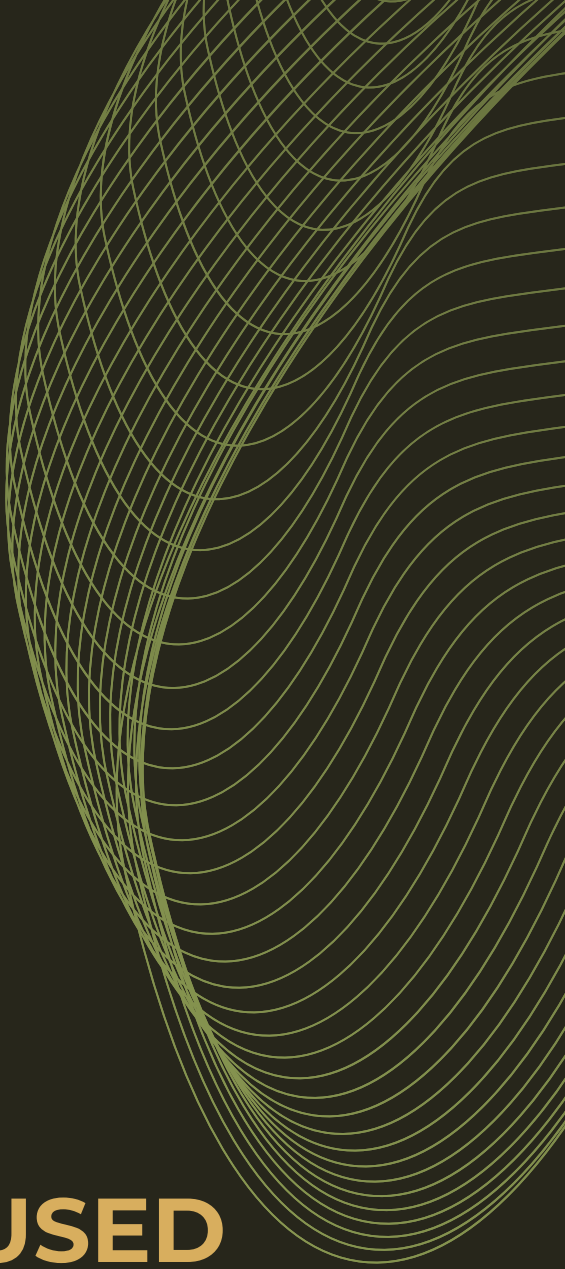
AMNA ASIF, MARIA
QASIM & TALHA EJAZ

Date: 13 dec, 2025



1. PROJECT OVERVIEW

The Word Search Puzzle Game is a console-based application implemented in C++ that allows users to play or automatically solve a standard word search puzzle. The core functionality involves generating a grid of letters with hidden words placed horizontally, vertically, or diagonally in all eight directions, and providing both manual (user-input) and automatic solving mechanisms.



DATA STRUCTURES USED

Grid (2D Array of Pairs)	Represents the game board. Each cell stores a pair<char, WORD>
Trie (Prefix Tree)	Stores all the words that are hidden in the grid.
Max Heap	Used during grid generation to prioritize words by length .
Vector	Used to store the list of words (words vector in Grid) and to implement the dynamic array for the heap (max_heap vector of pointers).

KEY FEATURES

- **Grid Management:** Generates and manages the letter grid, supporting word placement in all 8 directions (horizontal, vertical, diagonal).
- **Intelligent Placement:** Uses a Max Heap to prioritize placing longer words first, enhancing puzzle complexity.
- **User Solve Mode:** Allows manual coordinate input to find and color words.
- **Auto Solve Mode:** Automatically finds and colors all hidden words on the grid.
- **Visual Feedback:** Uses Windows Console API to color text, providing clear visual indication of found words.

WHAT'S NEW TO US

This project helped us learn several new concepts, including the use of advanced data structures to improve performance. We implemented a Max Heap to prioritize words based on length and used a Trie to store and validate dictionary words efficiently. We also learned how to use the Windows Console API to add colored text for better visual output. Additionally, the project strengthened our understanding of coordinate geometry by calculating directions for horizontal, vertical, and diagonal word paths.

ALGORITHMS USED :

Algorithm	Used In	Description
Max Heap	Maxheap	Maintains priority queues based on word length for efficient generation.
Trie	DictionaryTrie	Enables $O(L)$ (length-based) prefix checking and validation of words
Depth-First Search (DFS)	AutoSolve	Recursive pathfinding along fixed directions to match grid letters against the Trie dictionary.
Heuristic Placement	Grid_Generator	Strategically places longest words first to maximize successful grid population.

TIME ALLOCATION :

INVESTIGATION & RESEARCH: 3-4 DAYS

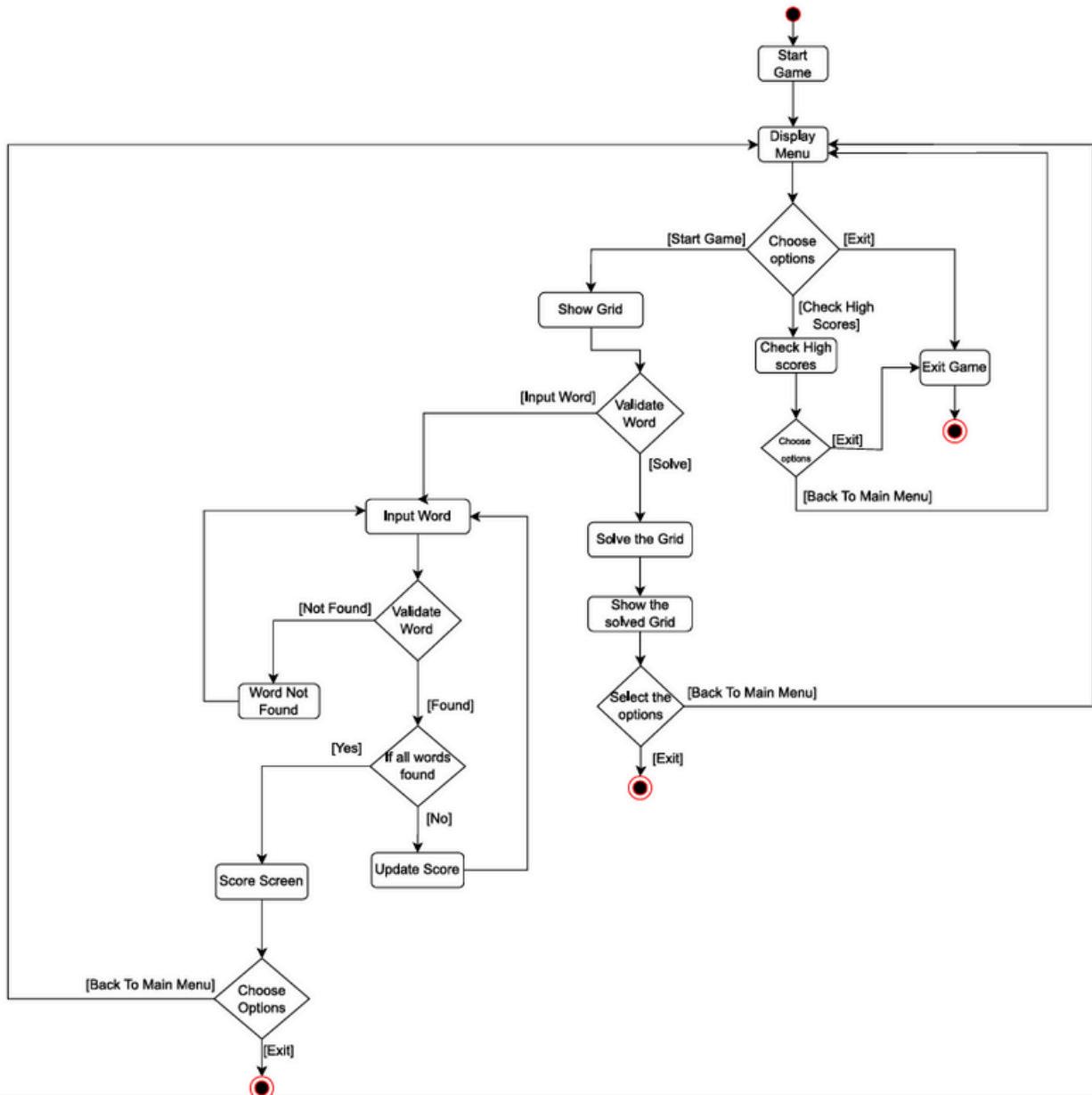
DESIGN & FLOWCHARTING : 5 HOURS

DOCUMENTATION : 3 HOURS

CODING & DEBUGGING : 2 WEEKS

TESTING : 24 HOURS

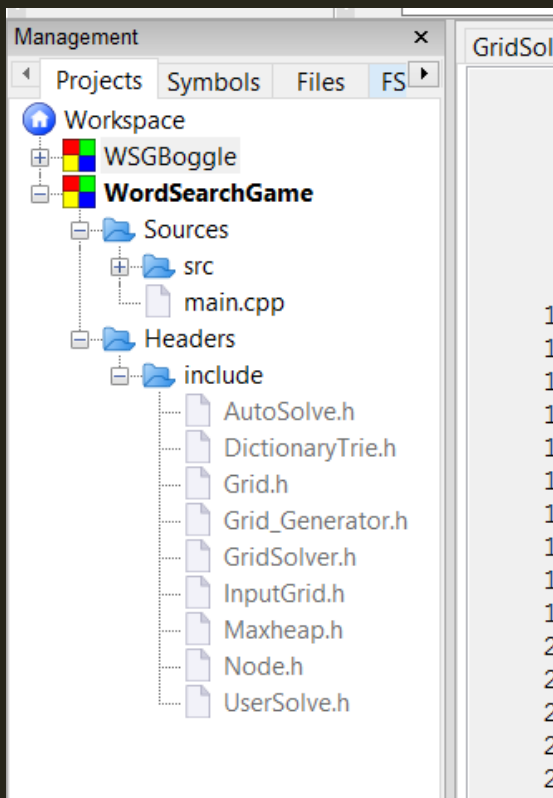
ACTIVITY DIAGRAM



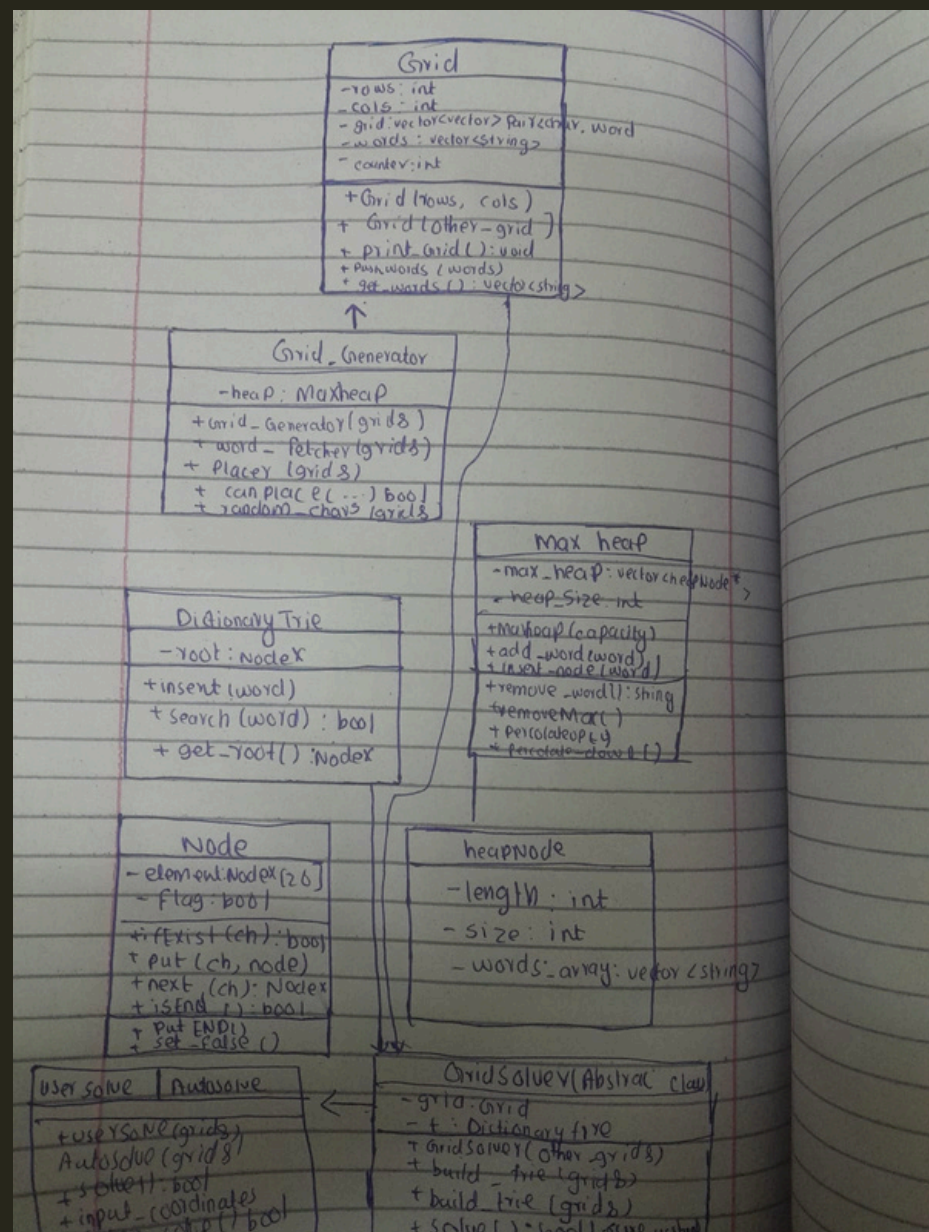
DIVISION OF WORK:

Component	Amna Asif	Maria Qasim	Talha Ejaz
I. Core Code Responsibility	Grid Structure & User Input Logic	Automatic Solver & Max Heap Structures	Grid Generation & Dictionary Structure
II. Primary Classes	Grid, UserSolve (Input Logic)	AutoSolve, Maxheap, heapNode, main	Grid_Generator, DictionaryTrie, Node, GridSolver (Base)
III. Code Functions	Grid definition, constructors, and display (print_Grid), User input handling	Auto-solve DFS logic, Max-Heap operations, main game control loop.	Word fetching, placement logic, Trie functions, Base class structure.

PROJECT DIRECTORY:



CLASS DIAGRAM:



OUTPUT :

```
"C:\Users\HP\OneDrive\Pictures\Documents\Semester No 3\DSA The
===== WELCOME TO THE GAME =====
1. Start Game.
2. Input Grid To Solve it.
3. Exit Game.
=====
Enter Command :
```

```
"C:\Users\HP\OneDrive\Pictures\Documents\Semester No 3\DSA Theo
===== GRID =====
===== Word Bank =====
if by it new far all
=====
u x f i o i r d
z r d b f g f n
n j d i y a i k
o e i a r o l b
k z w j s w y l
q p z r e l u n
c z v e w d g t
j e a b k v i r
=====
1. Solve by User
2. Auto Solve
3. Exit
Enter the command : _
```

```
"C:\Users\HP\OneDrive\Pictures\Documents\Semester No 3\DSA Theo
===== Word Bank =====
if by it new far all
===== SOLVE GRID =====
u x f i o i r d
z r d b f g f n
n j d i y a i k
o e i a r o l b
k z w j s w y l
q p z r e l u n
c z v e w d g t
j e a b k v i r
=====
Enter y/Y to proceed : _
```

```
"C:\Users\HP\OneDrive\Pictures\Documents\Semester No 3\DSA Theory\S
===== GRID =====
===== Word Bank =====
nor as on she you her
=====
p k l h n j d j
e.d l l p e o y i
y w m t e x r r t
o u n a c x k y
e w w e t s b p u
r o h r c s h o p
r s n u u l y o x
c g u p l m k s u
c
=====
1. Solve by User
2. Auto Solve
3. Exit
Enter the command : 1_
```


"C:\Users\HP\OneDrive\Pictures\Documents\Semester No 3\DSA Th

==== Word Bank =====
nor as on she you her

==== Grid =====

p k l h n j d j
d l l p e o y i
w m t e x r r t
o u n a c x k y
w w e t s b p u
o h r c s h o p
s n u u l y o x
g u p l m k s u

=====
Enter y/Y to proceed the game

Enter n/N to auto solve it : y

==== Command Section =====

Enter the 1st coordinate

Start Row : 6

Start Column : 0

Enter the 2nd coordinate

End Row : 4

End Column : 2_

"C:\Users\HP\OneDrive\Pictures\Documents\Semester No 3\DSA Theor

==== Word Bank =====
nor as on she you her

==== Grid =====

p k l h n j d j
d l l p e o y i
w m t e x r r t
o u n a c x k y
w w e t s b p u
o h r c s h o p
s n u u l y o x
g u p l m k s u

=====
Enter y/Y to proceed the game

Enter n/N to auto solve it : _